

Semantic Layer for Natural-Language Querying — Design Spec

Date: 2026-08-07 **Status:** Design approved pending user review **Source material:**
`C:\Users\legion\Downloads\export_3382893882042191526.pdf` (two posts: semantic metadata enrichment from SAP; On-Time Delivery KPI lineage)

1. Goal

Produce a single deliverable: a **step-by-step implementation guide for building a semantic layer whose success criterion is trustworthy natural-language querying**, accompanied by a small runnable reference implementation.

The guide must show, phase by phase, the **actual metadata artifact** produced at the end of each phase, carried across one continuous worked example with sample data — so a reader sees the same asset get progressively richer, and sees exactly which NL-query failure each phase eliminates.

1.1 Non-goals

- Not a LinkedIn post or article. (Ambiguity resolved: the user reframed the ask to "step by step implementation guide which can help in query in natural language".)
- Not a vendor product evaluation. The semantic model is tool-neutral YAML; portability notes are a short appendix, not a chapter.
- Not a governance/compliance implementation. Catalog metadata supports discovery; it is not a substitute for policy enforcement, consent management, access controls, or retention automation. The guide states this limit explicitly where governance fields appear.
- Not a text-to-SQL research survey. Exactly one architecture is specified and built (Section 4).

2. Organizing thesis

The semantic layer is not the destination. It is the grounding layer that makes natural-language querying trustworthy.

Every metadata phase in the guide justifies itself by closing one concrete NL failure mode:

Missing metadata	NL failure it causes
Business glossary + synonyms	Model cannot map "client number" to <code>party_key</code> ; question fails or binds to the wrong column
Declared grain + join cardinality	Split deliveries and rework loops silently double-count; answer is wrong and looks right
Governed metric definition	Model invents its own arithmetic; two users get two numbers for one question
Business process & event model	Model can answer <i>what</i> happened, never <i>why</i>
Data-quality contract	Answer computed from incomplete inputs is returned with false confidence
Published catalog entry	Answer arrives with no provenance, so no one can defend it in a review

The guide is structured so this table is *demonstrated*, not asserted: each phase's section ends with a before/after of the same NL question.

3. Domain and worked example

Domain: SAP Plan-to-Deliver — manufacturing through delivery. This extends the source posts' Order-to-Cash scope upstream into production, because the diagnostic NL question ("why did it drop?") is unanswerable without manufacturing phases.

3.1 Anchor KPI

On-Time Delivery % (`on_time_delivery_pct`) (orders delivered on or before promised delivery date / eligible orders due for delivery) × 100 Grain: one record per customer order. Exclusions: cancelled orders. Refresh: daily.

3.2 The two exemplar NL queries

These drive every design decision. Query 2 is the reason manufacturing is in scope.

The demo's as-of date is **2026-08-07**, so "last month" resolves to **2026-07**. June 2026 is the comparison window.

Query 1 — descriptive

"On-time delivery for India warehouses last month"

Resolves to: metric `on_time_delivery_pct` ; filters `region=IN` , `month=2026-07` ; grain `order` ; exclusions `cancelled` , `not-yet-due` . Returns:

87.3% — 48 of 55 eligible orders delivered on or before promised date. Provenance: `SAP SD` → `PP` → `QM` → `MM/EWM` → `TM` . Trust: `TRUSTED` .

`87.3%` is `48/55 = 87.2727%` , rounded to one decimal for display. Tests assert the unrounded ratio.

Query 2 — diagnostic

"Why did on-time delivery drop for India warehouses last month?"

Resolves to: same metric and filters, broken out by `delay_attribution_phase` , compared against the prior month. Returns:

87.3%, down 8.9pp from 96.2% in June (25 of 26 eligible). 7 orders were late. Attribution: **quality inspection 4**, transportation 2, production execution 1. On those 4 orders, QM usage decision ran an average **4.75 days over its 1.0-day standard, against 0 days of available slack** — the promise left no room to absorb it. Plant 1010.

Both answers are computed from the committed sample data (Section 5) and asserted in the acceptance criteria (Section 8), so every figure in the guide is reproducible rather than illustrative.

Query 1 needs only the glossary, grain, and metric definition. Query 2 additionally needs per-phase timestamps, per-phase planned baselines, and a variance-attribution rule **defined once in the semantic model**. That is the payoff for Phase 4, and the reason manufacturing is in scope.

3.3 Business process phases (manufacturing → delivery)

Twelve phases. Each is a row in `04_process_model.yml` , which carries the warehouse column that materializes the phase timestamp. The SAP fields below are the *business* provenance of each event and are illustrative: only those with a row in `01_technical_metadata.json` are harvested from DDIC, and `mrp_date` / `release_date` are synthesized by the generator rather than extracted, so they have no DDIC entry.

#	Phase	System / module	Business event	Timestamp field	Std. duration (days)
1	Demand & order capture	SAP SD	Sales order created	VBAK-ERDAT	0
2	Requirements planning	SAP PP (MRP)	Planned order created	PLAF-PSTTR	1
3	Production order release	SAP PP	Production order released	AFKO-GSTRP / GLTRP	1
4	Material staging & issue	SAP MM	Goods issue to production order	MSEG-BUDAT	1
5	Production execution	SAP PP	Operation confirmed	AFRU-BUDAT	3
6	Quality inspection	SAP QM	Usage decision (accept / reject / rework)	QALS-VDATUM	1
7	Goods receipt to stock	SAP MM / EWM	GR posted to unrestricted stock	MKPF-BUDAT	1
8	Picking & packing	SAP EWM	Picking completed	picking_completed_timestamp	1
9	Goods issue / dispatch	SAP LE-SHP	Actual goods issue	LIKP-WADAT_IST	0
10	Transportation	SAP TM	Shipment dispatched	VTTK-DPTBG	3
11	Delivery confirmation	SAP TM / LE	POD / delivery confirmed	delivery_timestamp	0
12	Billing	SAP SD	Invoice created	VBRK-FKDAT	1

3.4 Variance attribution model (makes "why" computable)

Per order, for each phase: `actual_duration_days` from consecutive event timestamps; `variance_days = actual_duration_days - standard_duration_days`.

```
delay_attribution_phase =
  argmax(variance_days) over the 9 variance spans (phase_seq 2..10),
  ties broken to the lowest phase_seq,
  evaluated only for orders where delivery_date > promised_delivery_date
```

Available slack per order = `promised_delivery_date - order_date - Σ standard_duration_days`.

This is defined in `05_semantic_model.yml` as a governed derived dimension, computed in SQL by the compiler. The LLM never derives it.

Phase 1 is the chain's start point rather than a span, phase 11 is a 0-day event folded into `var_shipment`, and phase 12 follows delivery — so 9 of the 12 phases are emittable attribution values. Declaring the other three would let a filter pass the gates and return zero rows dressed as an answer.

3.5 Governed metrics

Four, so the guide can show metric interaction and per-phase KPIs rather than a single toy measure.

- **Failure is refusal, not fabrication.** An unresolvable question fails the validator gate and returns "I can't answer that with governed metrics; here's what's missing" — instead of confident nonsense.

4.2 Validator gates

1. `metric` exists in the semantic model **and** `approval_state == approved`
2. Every `dimension` is declared for that metric's entity
3. Every `filter` column is bound to a real column and its value is in the declared allowed-values set (or passes the declared type/format)
4. `grain` matches the metric's declared grain
5. No requested join path introduces fan-out beyond the declared cardinality
6. `time_window` resolves to a concrete bounded range
7. Required exclusions from the metric definition are present in the compiled predicate

Gate failures return a structured refusal naming the failed gate.

4.3 Claude API contract

Grounded in the `claude-api` skill (read this session; do not answer these from memory when implementing):

Parameter	Value	Reason
<code>model</code>	<code>claude-opus-5</code>	Current default
Structured output	<code>client.messages.parse()</code> with a Pydantic <code>QueryIntent</code> model (<code>output_config.format</code> / <code>json_schema</code> under the hood)	Validated object, retried on schema mismatch; no hand-parsing
<code>thinking</code>	<code>{"type": "adaptive"}</code>	Synonym mapping + relative-date arithmetic is non-trivial
<code>output_config.effort</code>	<code>medium</code> , with a documented sweep note	Per-query latency matters; guide instructs readers to sweep <code>low</code> / <code>medium</code> / <code>high</code> on their own eval set
<code>max_tokens</code>	<code>16000</code>	Non-streaming; intent objects are small but thinking shares the budget
Prompt caching	<code>cache_control: {"type": "ephemeral"}</code> on the last system block (stable semantic-metadata context); volatile NL question in the user turn	Semantic context is reused across every query; Opus 5 minimum cacheable prefix is 512 tokens
Refusal handling	Check <code>response.stop_reason == "refusal"</code> before reading <code>content</code>	Required on Opus 5

Sampling parameters (`temperature` , `top_p` , `top_k`) and `budget_tokens` are **not** used — they return 400 on Opus 5.

4.4 Offline mode (required, not optional)

`ANTHROPIC_API_KEY` is not set in the target environment. The deliverable therefore ships `tests/golden_intents.json` — hand-authored `QueryIntent` objects for 12 canonical questions. Everything except the resolver runs with zero API calls: warehouse build, compiler, validator, provenance, all tests. `ask.py` takes `--offline` to compile a golden intent by question ID. The guide states this up front so a reader without a key still gets a working system.

5. Sample data

Two competing needs: percentages must be **believable** (87.3% is impossible with 6 orders — each order would move the number 16.7pp), and the interesting rows must be **readable**. Resolved by splitting the data into a hand-authored core and a

generated filler.

5.1 Scale

Windows are cut on `promised_delivery_date` , declared as the metric's time dimension. As-of date is 2026-08-07, so every July-promised order is due.

Segment	Orders in window	Cancelled	Eligible	On time	Late	OTD
IN — July 2026 (Query 1 & 2 target)	56	1	55	48	7	87.27%
IN — June 2026 (comparison)	27	1	26	25	1	96.15%
IN — August 2026	4	0	0 (all not-yet-due)	—	—	n/a
US — July 2026	30	2	28	26	2	92.86%
US — June 2026	15	0	15	14	1	93.33%
Total	132	4	124	113	11	—

The 7 late IN/July orders attribute as **quality inspection 4, transportation 2, production execution 1** — the answer Query 2 must produce.

The drop is `96.1538 - 87.2727 = 8.88pp`, reported as 8.9pp.

The August window carries the **not-yet-due** exclusion: 4 orders promised after the as-of date, so the eligible denominator is 0 and the metric must return "no eligible orders" rather than 0% or a divide-by-zero.

Dimension	Values
Plant	<code>1010</code> Pune, IN · <code>1710</code> Plano, US
Warehouse	<code>WH-IN-01</code> · <code>WH-US-01</code>
Carrier	<code>BLUEDART</code> · <code>FEDEX</code>
Region	<code>IN</code> · <code>US</code>

5.2 Hand-authored vs. generated

- `data/seed_core.csv` — **20 hand-authored orders**. Every defect in 5.3, every late order, and the full manufacturing event chain for each. These are the rows the guide prints and walks through line by line, and the rows the tests assert on by ID.
- `data/generate.py` — **deterministic filler**. Fixed seed (`random.Random(20260807)`), no wall-clock, no network. Produces the remaining 112 well-behaved on-time orders needed to hit the denominators above. Committed alongside its output CSVs, so the repo works without running it and the numbers never drift.

Every figure in the guide traces to committed CSVs. The generator makes the aggregates realistic; it does not make them arbitrary.

Raw extract files (SAP-shaped, in `data/`): `sales_orders_raw.csv` , `production_orders_raw.csv` , `inspection_lots_raw.csv` , `goods_movements_raw.csv` , `deliveries_raw.csv` , `shipments_raw.csv` .

5.3 Deliberate defects — each teaches one lesson

All in `seed_core.csv` , all in the IN / July window.

#	Defect	Where	Lesson it forces
1	Cancelled order	<code>SO-1007</code> , <code>order_status = CANC</code>	Exclusion must be in metadata, or the denominator inflates
2	Null <code>delivery_timestamp</code> , in transit, promised date in the future	<code>SO-1013</code>	Must not count as late; completeness rule needs a scoped predicate, not a blanket not-null
3	Rework loop — two QM usage decisions on one lot	<code>SO-1004</code> / lot <code>QL-4004</code>	Fan-out double-count; <code>first_pass_yield_pct</code> needs "first decision only" as a governed exclusion
4	Split deliveries — 6 eligible IN/July orders have 2 delivery rows each , incl. <code>SO-1009</code> → <code>DLV-2009A</code> (on time), <code>DLV-2009B</code> (late)	Grain double-count. Partial shipment is normal SAP behaviour, so this is the realistic trap, not a contrived one	
5	Promised date changed mid-flight	<code>SO-1011</code> (2026-07-18 → 2026-07-24)	Which promise is measured against? Forces an explicit SCD-2 / as-of decision in metadata
6	Make-to-stock order, no production order	<code>SO-1015</code>	LEFT JOIN semantics; manufacturing phases legitimately absent and must not null out the row or the metric
7	Late orders with QM-dominated variance	<code>SO-1002</code> , <code>SO-1004</code> , <code>SO-1006</code> , <code>SO-1010</code>	Produces Query 2's attribution answer, hand-verifiable from 4 printed rows
8	Late orders attributed elsewhere	<code>SO-1003</code> , <code>SO-1009</code> (transport) · <code>SO-1005</code> (production)	Attribution must discriminate, not always blame QM

5.4 The wrong answer, worked

Defects 3 and 4 are load-bearing: they are why declaring the grain stops being ceremony. The guide opens with the wrong answer and derives it explicitly.

An analyst joins `fact_order_delivery` to the delivery table and computes the metric at **delivery** grain instead of **order** grain. 6 of the 55 eligible IN/July orders shipped in two parts, so the row count becomes 61:

- 5 split orders delivered both legs on time (`SO-1012` , `SO-1014` , `SO-1016` , `SO-1017` , `SO-1018`) → 2 on-time rows each
- `SO-1009` delivered leg A on time and leg B late → 1 on-time row, 1 late row. At order grain it is **late** (the order isn't complete until the last leg lands).

Arithmetic:

	On time	Denominator	OTD
Grain-correct (order)	48	55	87.27%
Naive (delivery)	43 non-split + 10 split legs + 1 (<code>SO-1009</code> leg A) = 54	61	88.52%

The naive answer is **1.25pp too high** — and *higher* is the dangerous direction. The error flatters the number, so nobody questions it. It is close enough to pass review and close enough to put in a board deck.

Worse, the bias direction depends on the data: a fully on-time split order pulls the naive figure **up**, a fully late one pulls it **down**. So the error is not even a consistent offset you could learn to correct for. Grain mistakes do not announce themselves with absurd values; they quietly move the number by roughly the amount people argue about in review meetings. A validator that checks declared grain catches this. Eyeballing the output does not.

The same trap appears in `first_pass_yield_pct` : counting `SO-1004` 's rework re-inspection gives 52/55 = **94.55%** instead of the correct 51/54 = **94.44%** — again, wrong in the flattering direction.

`test_compiler.py` asserts every figure in this section, so the narrative is under test.

6. Build phases and their artifacts

Every phase section in the guide follows the same five-part shape: **INPUT** → **WORK** → **OUTPUT (a real file, shown in full or in a faithful excerpt)** → **GATE (how you know the phase is done)** → **NL FAILURE CLOSED (before/after on a real question)**.

Phase	Name	Artifact emitted	NL failure closed
P0	Scope & KPI contract	<code>00_kpi_contract.md</code> — the question inventory (12 NL questions the layer must answer), metric owners, decision the KPI supports	Building a semantic layer with no acceptance test for "done"
P1	Technical metadata harvest	<code>01_technical_metadata.json</code> — SAP DDIC extract: tables, fields, data elements, domains, keys, nullability, source system	Nothing to bind business terms to
P2	Standardize & model	<code>02_staging_model.sql</code> + <code>02_standardized_metadata.json</code> — canonical names, typed columns, conformed dimensions, grain declared per table	<code>VBELN</code> vs <code>vbeln</code> vs <code>sales_doc</code> — model can't tell they're one thing
P3	Business glossary & semantic binding	<code>03_glossary.yml</code> (terms, definitions, synonyms, allowed values, owner, steward) + <code>03_column_bindings.yml</code> (term → physical column)	Model can't map "client number" → <code>party_key</code> ; can't validate "India" as a region value
P4	Business process & event lineage	<code>04_process_model.yml</code> — 12 phases, systems, events, actors, timestamp bindings, standard durations, variance rules	Model answers <i>what</i> , never <i>why</i> ; no phase attribution possible
P5	Semantic model & metrics	<code>05_semantic_model.yml</code> — entities, dimensions, joins with cardinality , 4 metrics with grain/exclusions/business rules/approval state	Model invents arithmetic; double-counts on split deliveries and rework
P6	Quality contract	<code>06_dq_rules.yml</code> — rules per DQ dimension, each bound to a specific metric input, each with observed score vs. business expectation vs. enforced control	Confidently wrong answers from incomplete inputs
P7	Publish & NL resolution	<code>07_catalog_asset.json</code> (catalog/marketplace entry, lineage, governance fields) + the runnable resolver	Answers with no provenance; nothing to defend in a review

6.1 Phase 6 data-quality rules

Bound to metric inputs, so a failing rule degrades a specific answer rather than a dashboard-wide red light.

Dimension	Rule	Bound to
Completeness	<code>delivery_timestamp</code> not null where <code>promised_delivery_date < current_date</code> and <code>order_status <> 'CANC'</code> ; threshold ≤ 2% null	<code>on_time_delivery_pct</code> numerator
Validity	<code>delivery_status</code> ∈ { <code>DELIVERED</code> , <code>IN_TRANSIT</code> , <code>FAILED</code> , <code>RETURNED</code> }; <code>plant</code> ∈ { <code>1010</code> , <code>1710</code> }	filter binding
Uniqueness	exactly one row per <code>order_id</code> in <code>fact_order_delivery</code>	grain guarantee (defect 4)
Timeliness	mart <code>last_refreshed_at</code> within 26 hours	freshness badge on every answer
Consistency	<code>release_date</code> ≤ <code>gr_date</code> ≤ <code>goods_issue_date</code> ≤ <code>delivery_date</code>	variance attribution validity
Accuracy	recomputed <code>on_time_delivery_pct</code> within 0.1pp of certified monthly snapshot	metric certification

Each answer returns a **trust badge** — `TRUSTED` / `DEGRADED (rule X failing)` / `BLOCKED` — derived from these rules. A `BLOCKED` input returns a refusal, not a number.

6.2 Phase 7 governance fields (with stated limits)

`07_catalog_asset.json` carries: owner, steward, data-subject category, personal-data category, processing purpose, legal basis, access group, retention period, masking policy, certification state, lineage chain, related metrics.

The guide states plainly: these fields make personal data **discoverable** and support India DPDP readiness work. They do not enforce anything. Enforcement lives in access control, consent management, retention automation, and masking at the platform layer.

7. Deliverable structure

semantic-layer-nl/	
README.md	the guide (primary deliverable)
data/	
seed_core.csv	20 hand-authored orders (all defects, all late orders)
generate.py	deterministic filler, fixed seed 20260807
sales_orders_raw.csv	
production_orders_raw.csv	
inspection_lots_raw.csv	
goods_movements_raw.csv	
deliveries_raw.csv	
shipments_raw.csv	
metadata/	
00_kpi_contract.md	
01_technical_metadata.json	
02_standardized_metadata.json	
03_glossary.yml	
03_column_bindings.yml	
04_process_model.yml	
05_semantic_model.yml	
06_dq_rules.yml	
07_catalog_asset.json	
sql/	
02_staging_model.sql	
src/	
build_warehouse.py	raw CSV → staging → marts (DuckDB)
semantic/	
loader.py	parse + structurally validate the YAML model
retriever.py	select the metadata slice for a question
resolver.py	Claude API: NL → QueryIntent (json_schema)
validator.py	the 7 gates
compiler.py	QueryIntent → SQL (deterministic)
provenance.py	answer + lineage + trust badge
dq.py	run 06_dq_rules.yml, emit badge
ask.py	CLI: python ask.py "... " [--offline ID]
tests/	401 tests, 3 skipped without an API key
golden_intents.json	Q1 and Q2 (offline mode); see 10.2
test_data.py	Section 5.1 denominators vs. committed CSVs
test_metadata.py	every artifact parses, no placeholders
test_warehouse.py	staging + marts, attribution SQL
test_loader.py	the model parses and structurally validates
test_retriever.py	unapproved metrics are never offered
test_validator.py	each gate rejects what it must
test_compiler.py	grain / fan-out / exclusion regressions
test_provenance.py	lineage chain and trust badge
test_dq.py	each defect trips its intended rule
test_end_to_end.py	CLI answers Q1 and Q2 as specified
test_diagram.py	all 3 diagrams match the code they depict
test_guide.py	README claims vs. artifacts and warehouse
test_content.py	all three published documents, same treatment
test_standalone.py	the single-file demo runs, and agrees with the project it was extracted from

test_bundle.py	the "everything" PDF really is everything
test_resolver_live.py	skipped without ANTHROPIC_API_KEY
docs/	
design.md	this document
md2pdf.py	markdown → styled HTML → PDF (headless Chrome)
build_bundle.py	all 5 documents + all 9 artifacts → one PDF
semantic-layer-complete.md	the assembled bundle, regenerated not edited
semantic-layer-complete.pdf	the same, printed (no page count stated here: the bundle contains this file, so any number written here changes the number)
diagrams/	
render_dataflow.py	renders the swimlane [--check]
nl_dataflow.svg / .png	embedded in the guide
render_sources.py	renders the provenance map [--check]
metadata_sources.svg / .png	embedded in the guide
render_grain.py	renders the two-counting-rules figure that opens the Medium post -- no jargon, one order, three hand-counted rows [--check]
grain_two_ways.svg / .png	the Medium post's lead image
content/	the three published deliverables
linkedin-post.md	part 1, ~250 words
linkedin-article.md	part 2, the long-form walkthrough
medium-post.template.md	part 3's prose, with {{...}} splice markers
medium-post.md	part 3, generated -- never hand-edited
medium-publishing-checklist.md	title, tags, images, canonical URL, sequencing: the working notes, kept out of the article
build_medium_post.py	splices real code and real output into the template, so the published code cannot drift from the tested code
medium-kit.md	part 3 as a publication kit, generated: cover, the article with the code lifted out, checklist
medium-code.md	the lifted code on its own, with its sha256
build_medium_kit.py	derives both from medium-post.md, so the split copies cannot drift from the source either
*.html / *.pdf	each part rendered for review; the HTML is compared byte-for-byte against a fresh render, so a stale copy fails the suite
standalone/	
semantic_layer_demo.py	the whole architecture in one runnable file: embedded data, inline metadata, 7 gates, compiler, diagnostic. pip install duckdb pyyaml
appendix/	
portability.md	mapping to dbt MetricFlow, Cube, Databricks Unity Catalog, Atlan, Collibra, SAP Datasphere
verify_acceptance.py	measures Section 8, one verdict per criterion

7.1 Guide reading order

1. Why NL querying fails without a semantic layer (the two exemplar queries, wrong answers first)
2. The Plan-to-Deliver process and the anchor KPI
3. Phases P0–P7, one section each, artifact shown
4. The NL resolution path end to end (retriever → resolver → validator → compiler → provenance)

5. Running it yourself (with and without an API key)
6. What this does **not** do (governance limits, scale limits, single-fact-table limits)
7. Appendix: portability

8. Acceptance criteria

The deliverable is done when all of the following hold:

1. `python src/build_warehouse.py` builds the DuckDB warehouse from the committed CSVs with no manual steps and no network access.
2. `python src/ask.py --offline Q1` returns **87.3%** (48/55) for India / 2026-07 with the full provenance chain and a **TRUSTED** badge.
3. `python src/ask.py --offline Q2` returns **87.3%, -8.9pp vs. 96.2%**, with attribution **quality inspection 4, transportation 2, production execution 1**.
4. `pytest` passes with **no ANTHROPIC_API_KEY** set; live resolver tests skip cleanly.
5. `test_compiler.py` asserts the grain-correct **87.27% (48/55)** and the naive delivery-grain **88.52% (54/61)** from Section 5.4, plus FPY **94.44% (51/54)** vs. naive **94.55% (52/55)** — the "wrong answer first" narrative is under test, including that both errors overstate.
6. `test_dq.py` proves each of the 8 seeded defects trips its intended rule and no others.
7. `test_data.py` asserts every denominator in Section 5.1 against the committed CSVs, so spec and data cannot drift.
8. `python data/generate.py` is deterministic: re-running it leaves the CSVs byte-identical.
9. Every one of the 9 metadata artifacts exists as a real, parseable file — no placeholders, no **TBD**. (Nine files across eight phases: P3 emits both `03_glossary.yml` and `03_column_bindings.yml`.)
10. Each phase section in `README.md` names the specific NL failure it closes, with a before/after.
11. The governance-limits statement appears in the guide body, not buried in an appendix.
12. With a valid `ANTHROPIC_API_KEY`, `python src/ask.py "on-time delivery for India warehouses last month"` produces an intent matching golden intent `Q1`.

Two criteria were added after the spec's original list of 12. They are numbered 12 and 13 by `verify_acceptance.py` and labelled additive rather than renumbering the list above, so the spec's numbering still matches the plan it came from:

- **12 (additive)**. The dataflow diagram is committed, current, and embedded: `docs/diagrams/nl_dataflow.svg` and `.png` exist, `render_dataflow.py --check` exits 0, and `README.md` embeds the SVG.
- **13 (additive)**. The metadata sourcing map is committed, current, embedded — and *artifact-backed*: every one of its 13 source→artifact edges carries a **probe** string that must occur in the artifact it points at, and the figure's central claim (exactly one machine-harvestable artifact) still holds. A provenance diagram is the one kind that nothing else can catch lying, since "this came from SAP DDIC" is unfalsifiable unless something checks the file.

The third figure, `grain_two_ways`, has no acceptance criterion — it was added for the Medium post's plain-English opening, after the criteria were settled, and renumbering to accommodate it would break the correspondence above. `tests/test_diagram.py` covers it to the same standard: every figure in it is recomputed from the warehouse and from the standalone demo's embedded rows, so the friendly simplification cannot drift into a lie.

9. Where this lives — settled

`C:\Users\legion` is not a git repository, so committing this design document needed a home. Three options were on the table: leave it uncommitted, `git init` a specs-only repo under `C:\Users\legion\docs\`, or give the deliverable its own repo and move the spec into it.

Resolved as (c). The deliverable is a self-contained runnable project and benefits from its own history, and a spec that travels with the code it specifies is the one that gets updated when the code changes. This document is `docs/design.md` in

C:\Users\legion\semantic-layer-nl\ , built on branch `feat/semantic-layer-mvp` .

10. MVP scope

The first increment must be **runnable and honest**, not fully featured. Definition of the MVP: *a reader clones the project, runs two commands with no API key, and gets the guide's two headline answers with correct provenance — and the tests prove the numbers.*

10.1 In the MVP

Area	MVP content
Metadata artifacts	All 8. These are the deliverable — the guide is <i>about</i> the artifact each phase emits. Cutting any would gut the thesis.
Business process	All 12 phases in <code>04_process_model.yml</code> , with SAP field bindings and standard durations
Metrics	3 — <code>on_time_delivery_pct</code> (full narrative), <code>mfg_schedule_adherence_pct</code> (manufacturing KPI), <code>first_pass_yield_pct</code> (proves a second grain + the rework exclusion)
Sample data	Full Section 5 dataset — <code>seed_core.csv</code> + generator, all 8 defects
Pipeline	<code>build_warehouse.py</code> , <code>loader</code> , <code>validator</code> , <code>compiler</code> , <code>provenance</code> , <code>dq</code>
NL path	Offline via golden intents for Q1 and Q2 ; <code>resolver.py</code> present and callable, exercised only when a key exists
Tests	<code>test_compiler.py</code> , <code>test_dq.py</code> , <code>test_data.py</code> — acceptance criteria 1–11
Guide	<code>README.md</code> covering P0–P7 with each phase's artifact and the NL failure it closes

10.2 Deferred past the MVP

Deferred	Why it's safe to defer
<code>production_cycle_time_days</code>	A plain average over columns the other three metrics already materialize. Cheapest later addition, least instructive to build now.
Golden intents beyond Q1/Q2 (10 of 12)	Q1 and Q2 exercise every component: descriptive path, diagnostic path, grain, exclusions, attribution. The other 10 add coverage, not capability.
Live-resolver eval / effort sweep	Cannot be run or verified in this environment (no API key). Documented as instructions, not asserted as results.

Two rows that were on this list are no longer deferred, and are recorded here because what changed is worth keeping:

Was deferred	What actually shipped
<code>appendix/portability.md</code> as a stub table	Ships as the full mapping: every concept across dbt MetricFlow, Cube, Unity Catalog and Datasphere, a second table for Atlan and Collibra, and the three gaps a platform move inherits (grain is rarely first-class, <code>allowed_values</code> rarely enforces, <code>approval_state</code> is a tag almost everywhere).
<code>test_validator.py</code> per-gate rejection tests	Ships with 22 tests. Deferring these turned out to be the wrong call: writing them found that <code>exclusions_applied</code> — alone among the seven — had never been observed refusing anything. It was implemented, registered, and asserted <code>True</code> on the happy path, which is indistinguishable from a gate that always passes. It now has four rejection tests (dropped block, empty list, missing <code>predicate</code> , missing <code>key</code>), plus one that fails if any future gate is added without a rejection test.

10.3 MVP acceptance

Acceptance criteria 1–11 in Section 8 apply as written. Criterion 12 (live resolver) is documented but **not verifiable here** — the guide will state that plainly rather than imply it was tested.

11. Decisions already settled

Recorded so they are not relitigated during implementation:

Decision	Choice
Artifact type	One implementation guide + runnable reference implementation. Not a post, not three documents.
Success criterion	Trustworthy NL querying
Domain	SAP Plan-to-Deliver (manufacturing through delivery), continuing the source posts' narrative
"Phases of metadata"	The metadata artifact emitted at the end of each build phase; the same asset gets progressively richer
NL layer approach	Tool-neutral YAML semantic model plus runnable demo (DuckDB + Claude API + validation gate)
Query generation	NL → validated intent JSON → deterministic compiler. The model never writes SQL.
Sample data	132 orders (20 hand-authored + deterministic filler), 2 plants / 2 warehouses / 2 carriers, 8 deliberate defects. Sized so 87.3% and 96.2% are real, not illustrative.
Offline operation	Required — golden intents make everything but the resolver runnable without an API key
Model	<code>claude-opus-5</code> , adaptive thinking, <code>messages.parse()</code> with a Pydantic intent schema
Build strategy	MVP first (Section 10): runnable end to end with no API key, 3 metrics, all 8 artifacts, Q1 + Q2 answering correctly under test